

12-数据库

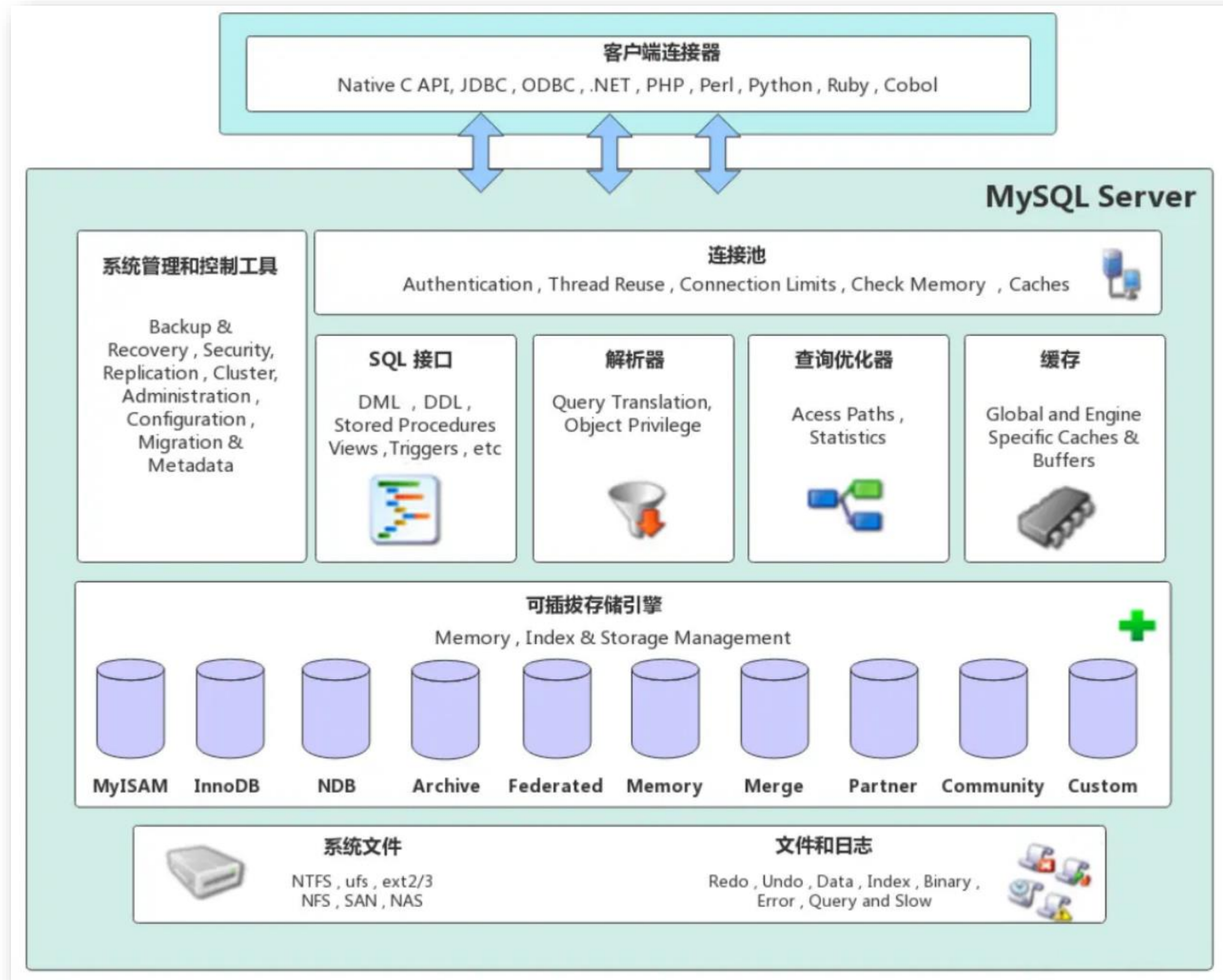
刘亚雄

极客时间-Java 讲师



一、MySQL数据库架构

1.1 MySQL逻辑架构图



- ① 客户端连接器
- ② 系统管理和控制工具
- ③ 连接池
- ④ SQL接口
- ⑤ 解析器
- ⑥ 查询优化器
- ⑦ 缓存
- ⑧ 存储引擎
- ⑨ 文件和日志

Redo log 、Undo log、Error Log、SlowQueryLog...

1.2 MySQL的日志文件

MySQL从物理结构上可以分为**日志文件**和**数据及索引文件**，MySQL通过日志记录了数据库操作信息和错误信息。

常见日志文件如下：

- ① 错误日志error.log
- ② 二进制日志mysql-bin
- ③ 通用查询日志general_query.log
- ④ 慢查询日志slow_query_log.log
- ⑤ 重做日志Redolog
- ⑥ 中继日志relaylog
- ⑦ Undo log
- ⑧ ...

如何知道这些日志文件在哪里？

```
1 | mysql> show variables like 'log_%';
```

```
mysql> show variables like 'log_%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_bin       | OFF   |
| log_bin_basename |      |
| log_bin_index  |      |
| log_bin_trust_function_creators | OFF   |
| log_bin_use_v1_row_events | OFF   |
| log_built_in_as_identified_by_password | OFF   |
| log_error      | /var/log/mysqld.log |
| log_error_verbosity | 3     |
| log_output     | FILE  |
| log_queries_not_using_indexes | OFF   |
| log_slave_updates | OFF   |
| log_slow_admin_statements | OFF   |
| log_slow_slave_statements | OFF   |
| log_statements_unsafe_for_binlog | ON    |
| log_syslog     | OFF   |
| log_syslog_facility | daemon |
| log_syslog_include_pid | ON    |
| log_syslog_tag  |      |
| log_throttle_queries_not_using_indexes | 0     |
| log_timestamps | UTC   |
| log_warnings   | 2     |
+-----+-----+
21 rows in set (0.00 sec)
```

1.2 MySQL的日志文件详解

01-错误日志error log

- 默认开启，记录运行过程中**所有严重的错误信息**，及每次启动和关闭的详细信息
- 通过log_error和log_warnings配置
 - log_error：指定存储位置
 - log-warnings：配置警告信息级别，0值不记录告警日志，大于等于1值记录告警日志

```
1 log_error=/var/log/mysql-error.log
2 log_warnings=2
```

02-二进制日志bin log

- 默认关闭，需通过配置开启，记录数据库所有的DDL语句和DML语句，不包括DQL语句
- 主要用于实现主从复制、数据备份、数据恢复
- 通过log-bin配置

```
1 server_id=42
2 log-bin=mysql-bin
```

1.2 MySQL的日志文件详解

03-通用查询日志

- 默认关闭，记录用户的所有操作，包含增删查改等详细信息。如果不是调试，不建议开启
- 通过general_log和general_log_file配置
 - general_log：日志开关
 - general_log_file：日志存储路径，如果没指定默认host_name.log

```
1 general_log={ON|OFF}
2 general_log_file=/var/lib/mysql/host_name.log
```

04-慢查询日志

- 默认关闭，记录SQL执行时间超阈值时间的所有查询
- 通过slow_query_log、long_query_time和slow_query_log_file配置
 - slow_query_log：日志开关
 - long_query_time：慢查询阈值
 - slow_query_log_file：日志存储路径， 如果没指定默认host_name-slow.log

```
1 slow_query_log=ON
2 long_query_time=10
3 slow_query_log_file=slow_query_log.log
```

1.3 MySQL数据文件

MySQL的数据及索引文件存储在哪里？

```
1 | show variables like '%datadir%';
```

情况01-ibdata文件：

- 使用**系统表空间**存储表数据和索引信息，那么数据存储在一个或多个ibdata文件中

情况02-InnoDB存储引擎的数据文件

- 表名.frm文件：主要存放与表相关的元数据信息，包括：**表结构的定义信息**
- 表名.ibd文件：一张表一个ibd文件，存储表数据和索引信息

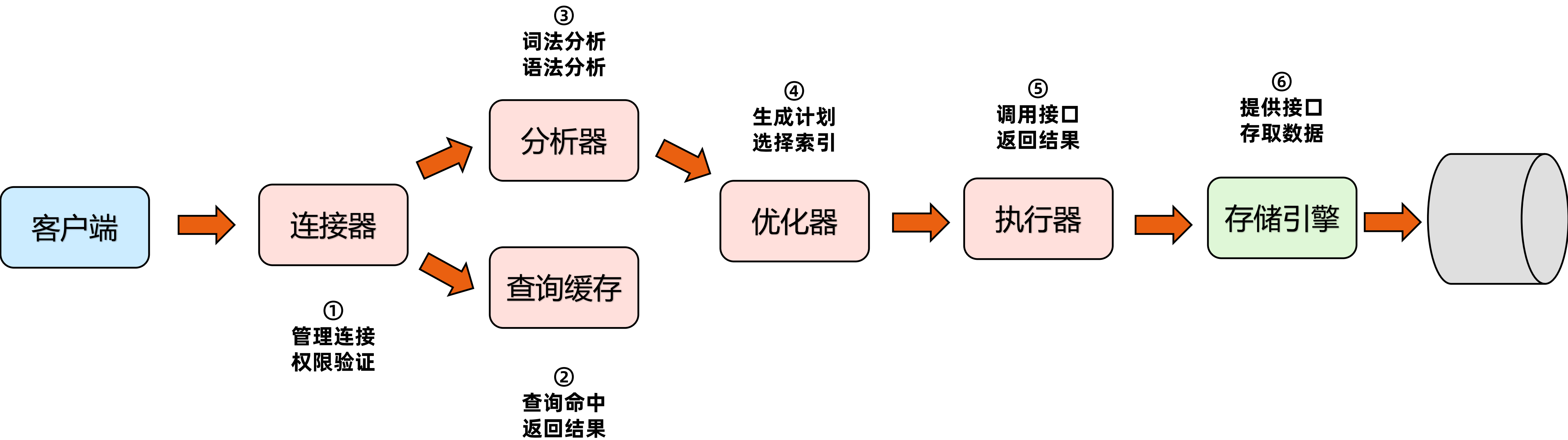
情况03-MyISAM存储引擎的数据文件

- 表名.frm文件：主要存放与表相关的元数据信息，包括：**表结构的定义信息**
- 表名.myd文件：主要存放数据
- 表名.myi文件：主要存放索引

1.4 一条SQL语句的完整执行流程

分析SQL语如下

```
1 | select c_id,first_name,last_name from customer where c_id=14;
```



 看课程笔记中过程的详细剖析

二、MySQL的存储引擎之InnoDB

2.1 MySQL存储引擎

01-存储引擎种类

- InnoDB
- MyISAM
- Memory
- Falcon
- ...

Engine	Support	Comment	Transactions	XA	Savepoints
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL

02-如何选择存储引擎？

- 除非需要用到某些InnoDB不具备的特性，并且没有其他办法可以替代，否则都应该选择InnoDB引擎

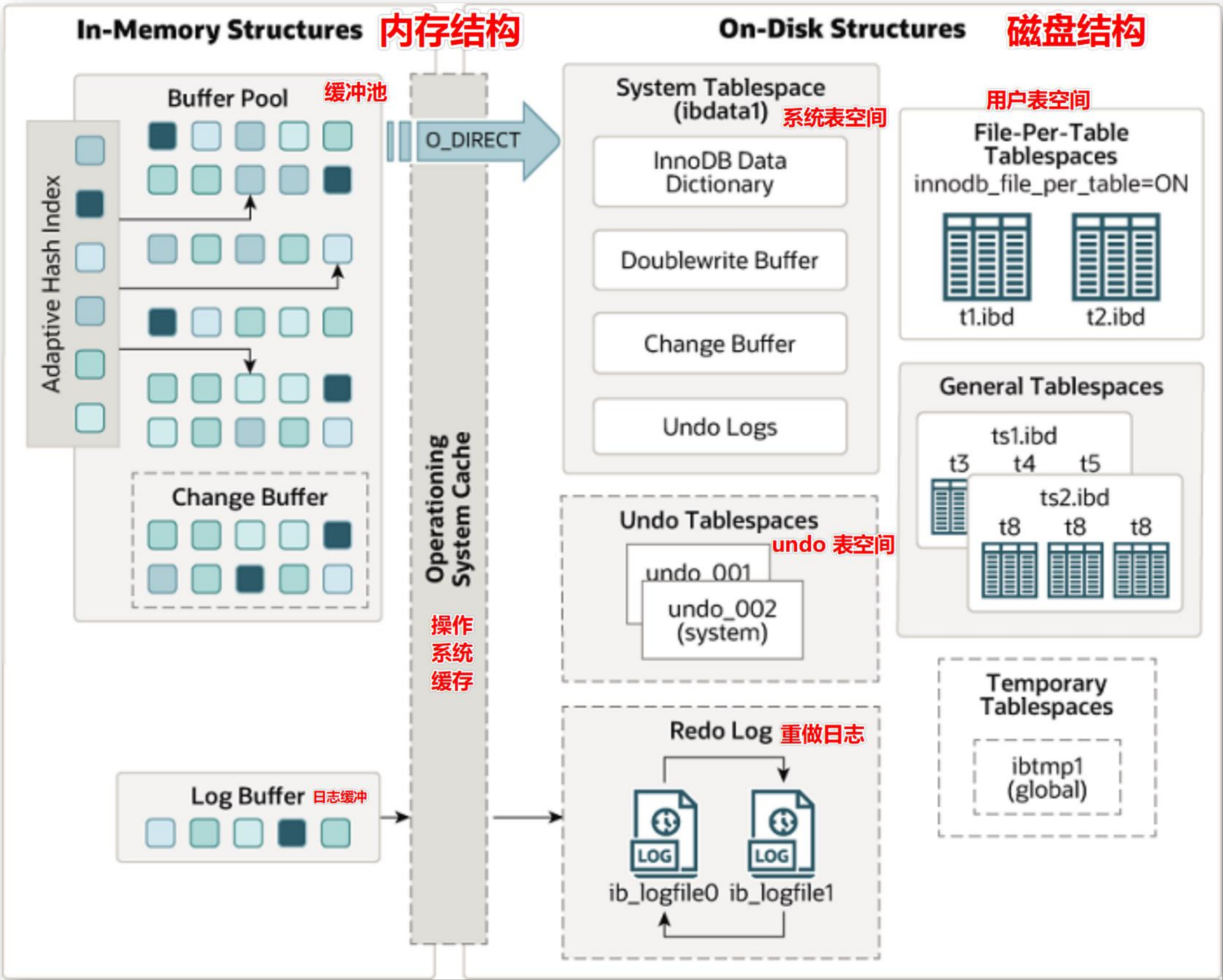
03-InnoDB VS MyISAM

比较	InnoDB	MyISAM
存储文件	.frm 表定义文件 .ibd 数据文件和索引文件	.frm 表定义文件 .myd 数据文件 .myi 索引文件
锁	表锁，行锁	表锁
事务	支持	不支持
CRUD	读写均可，系统崩溃后数据自动恢复	读多写少，崩溃后数据无法安全恢复
索引结构	B+ Tree	B+ Tree



如何查看和设置存储引擎？

2.2 InnoDB架构图



01-内存结构

- 1. Buffer Pool 缓冲池
- 2. Change Buffer 修改缓冲
- 3. Adaptive Hash Index 自适应哈希索引
- 4. Log Buffer 日志缓冲

02-磁盘结构

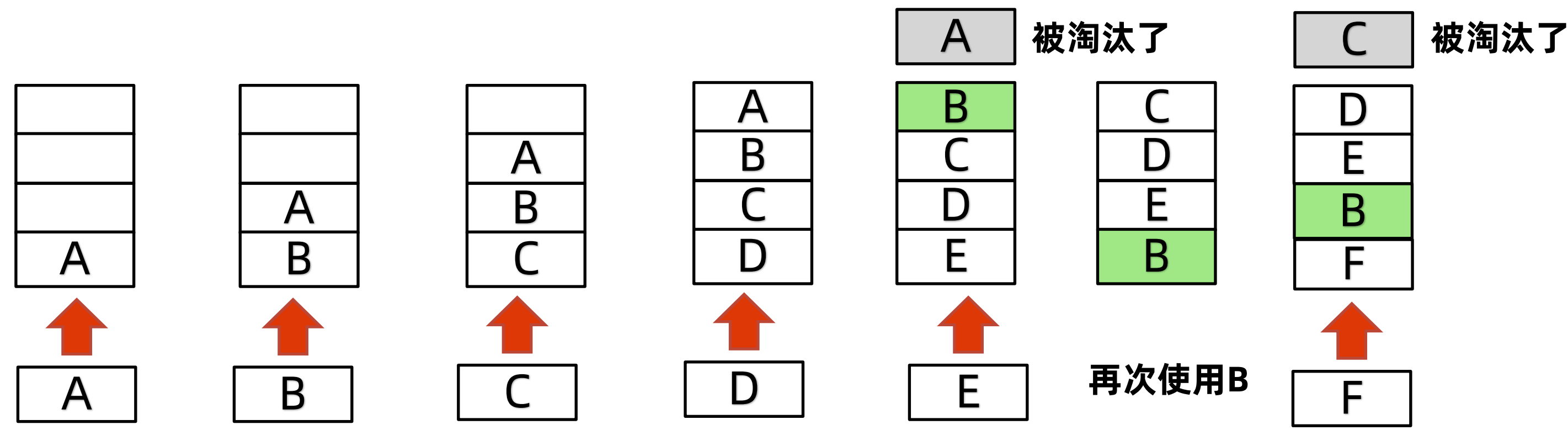
- 系统表空间System Tablespace
- 独立表空间File-per-table TableSpace
- 通用表空间General Tablespace
- 临时表空间Temporary Tablespace
- ...

💡 接下来，我们看一下内存结构

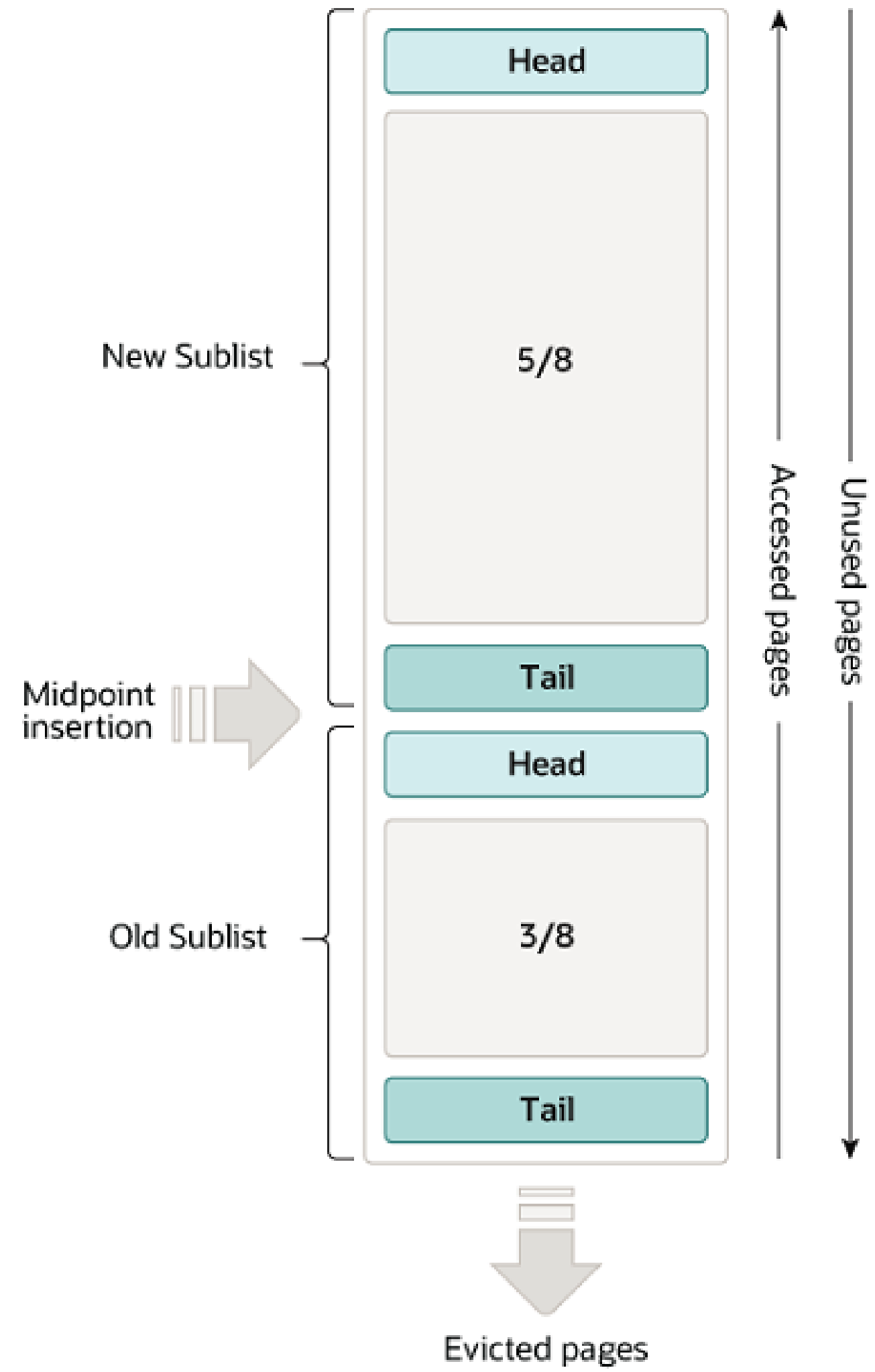
2.3 InnoDB-内存结构

01-内存结构-缓冲池Buffer Pool

- 缓冲池Buffer Pool 用于加速数据的访问和修改
- 默认大小为128M
- 缓存数据到内存，最大限度地减少**磁盘 IO**，加速**热点数据**的读和写
- 由于内存有限，Buffer Pool 仅能容纳最热点的数据
- 使用LRU算法（最近最少使用）淘汰非热点数据页：
 - ◆ LRU：根据页数据的历史访问来淘汰数据，**如果数据最近被访问过，那么将来被访问的几率也更高**，优先淘汰最近没有被访问到的数据。
- Buffer Pool中的数据**以页为存储单位**，数据结构是**单链表**



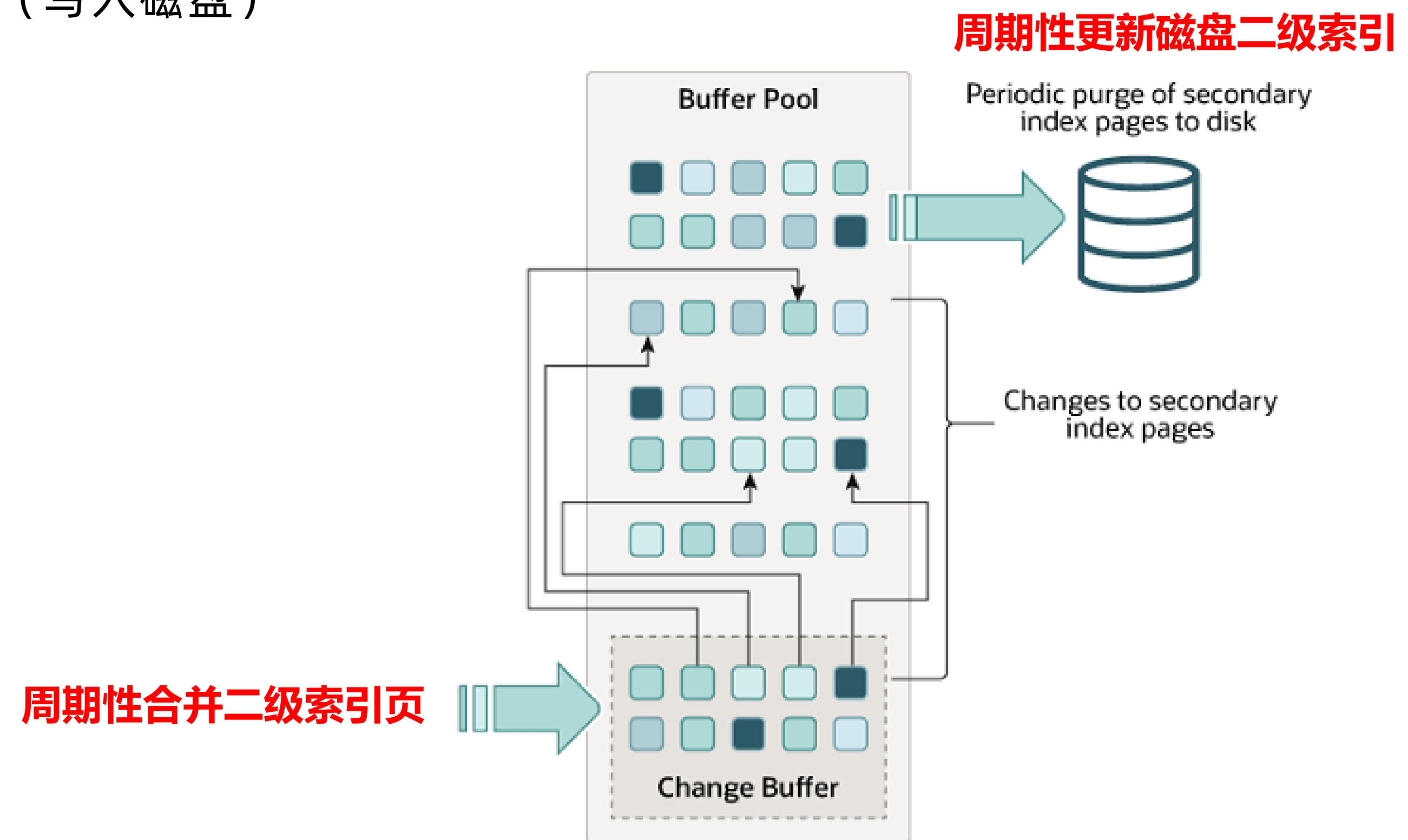
LRU算法（Least Recently Used最近最少使用）



2.3 InnoDB-内存结构

02-内存结构-修改缓冲Change Buffer

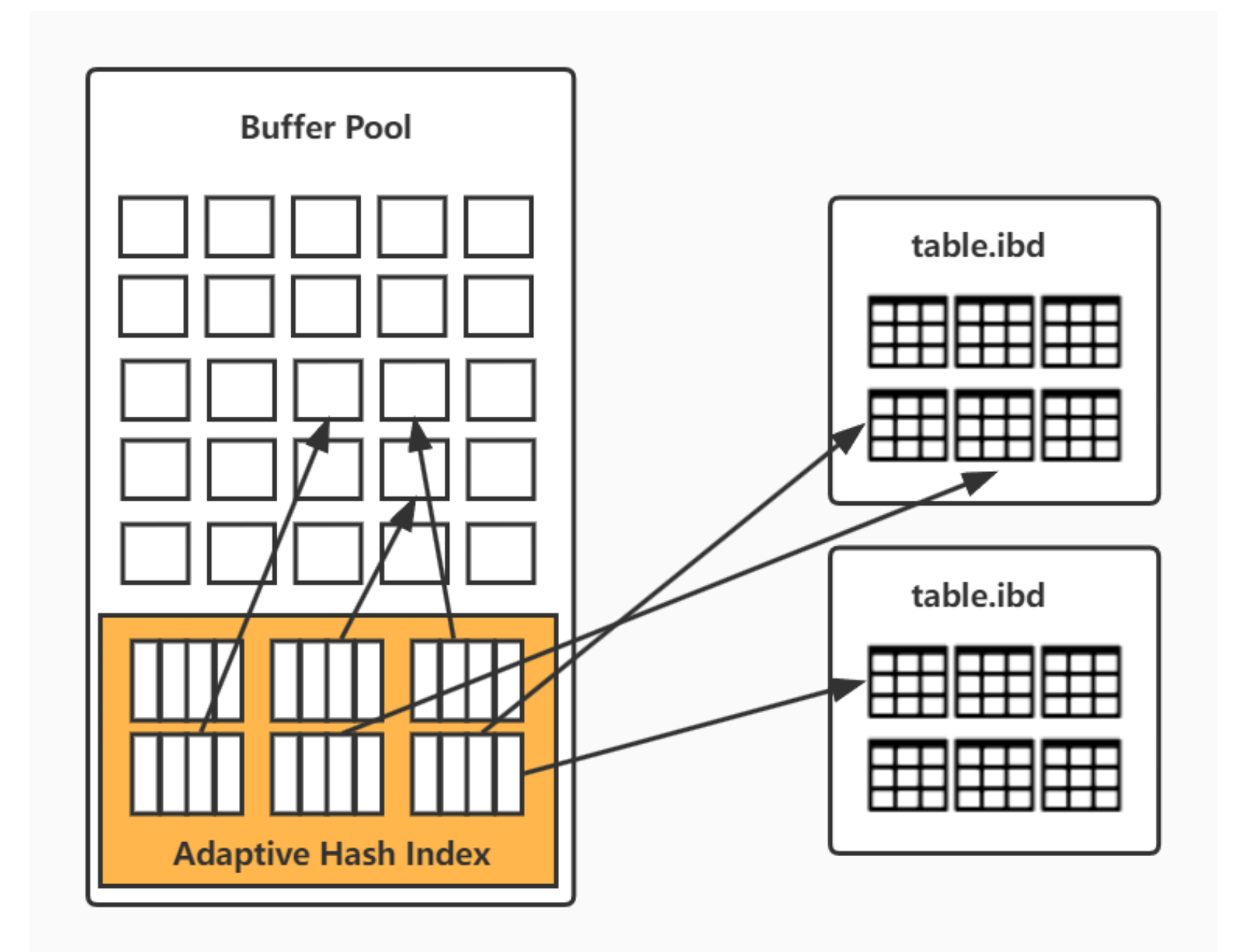
- 用于加速**非热点数据**中二级索引的写入操作
- 修改缓冲对二级索引的修改操作会录入 redo log 中
- 在缓冲到一定量或系统较空闲时进行merge操作（写入磁盘）
- 修改缓冲在系统表空间中有相应的持久化区域
- 其物理结构为一棵名为 ibuf 的 B+树



2.3 InnoDB-内存结构

03-内存结构-自适应哈希索引Adaptive Hash Index

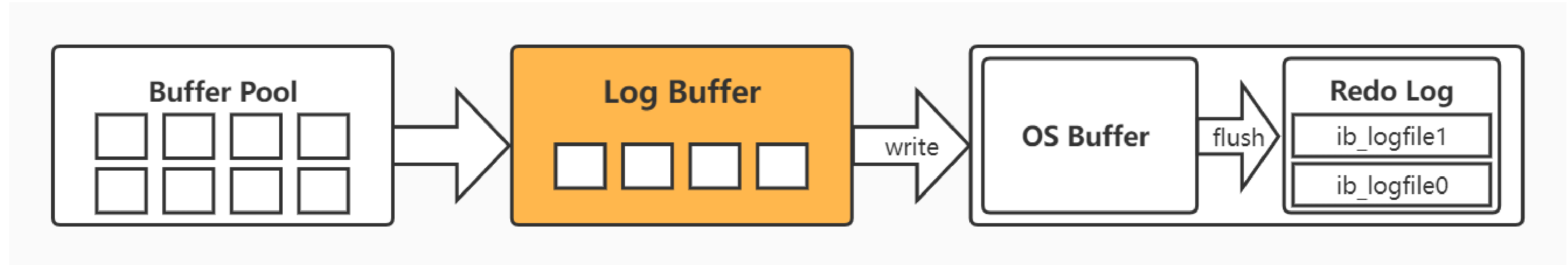
- 自适应哈希索引（Adaptive Hash Index, AHI）用于实现对于热数据页的一次查询，**是建立在索引之上的索引**
- **作用：对频繁查询的数据页和索引页进一步提速**
- AHI大小为Buffer Pool的1/64
- **若二级索引命中AHI：**
 - 从AHI获取索引页记录指针，再根据主键沿着聚簇索引查找数据
- **若聚簇索引命中AHI：**
 - 则直接返回目标数据页的记录指针，根据记录指针可以直接定位数据页



2.3 InnoDB-内存结构

04-内存结构-日志缓冲Log Buffer

- InnoDB 使用 Log Buffer 来缓冲日志文件的写入操作
- **内存写入** 加上 **日志文件顺序写**使得 InnoDB 日志写入性能极高。



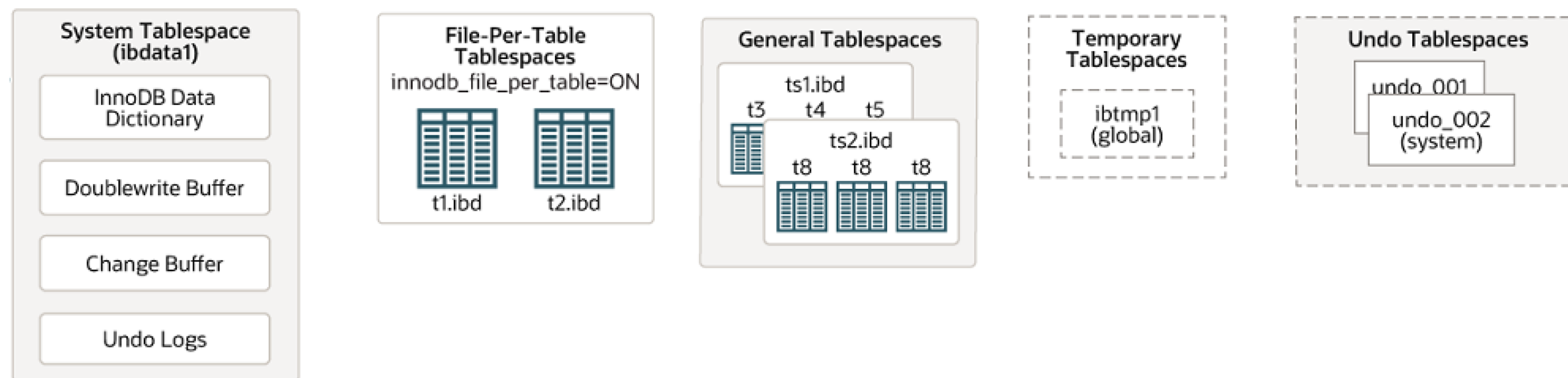
这种操作会带来什么问题？

2.4 InnoDB-磁盘结构

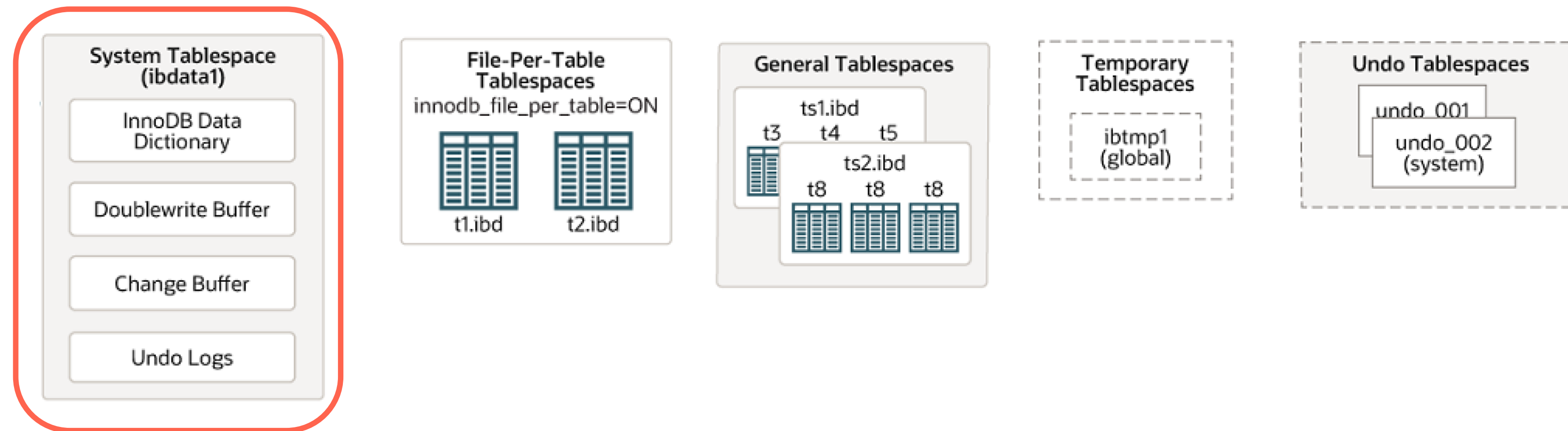
在磁盘中，InnoDB存储引擎将数据、索引、表结构和其他缓存信息等存放的空间称为表空间（Tablespace）它是物理存储中的最高层，由段Segment、区extent、页Page、行Row组成。

常见表空间：

- 系统表空间（System Tablespace），关闭独立表空间，所有表数据和索引都会存入系统表空间
- 独立表空间（File-per-table Tablespace），开启独立表空间，每张表的数据都会存储到一个独立表空间
- 通用表空间（General Tablespace）
- 回滚表空间（Undo Tablespace）
- 临时表空间（The Temporary Tablespace）



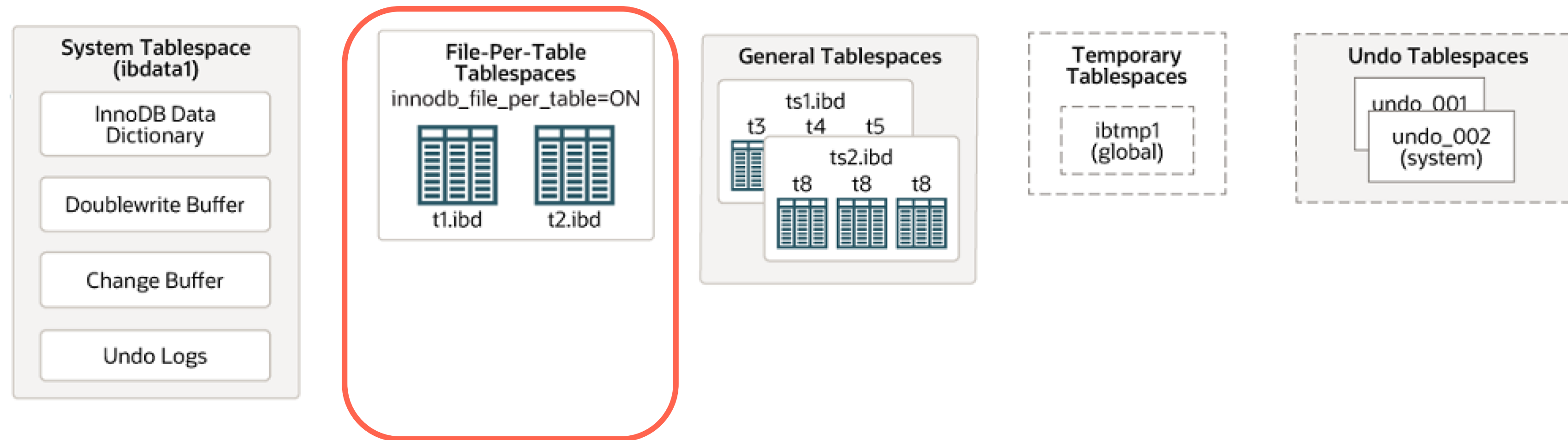
2.4 InnoDB-磁盘结构



01-系统表空间（System Tablespace）

- 系统表空间是 InnoDB 数据字典、**双写缓冲**、**修改缓冲**和**回滚日志**的存储位置如果关闭独立表空间，它也将存储所有表数据和索引。
 - 数据字典：表对象的元数据信息（表结构、索引、列信息等等）
 - **双写缓冲**：保证写入数据时，页数据的完整性，防止部分写失效问题
 - **修改缓冲**：内存中Change Buffer对应的持久化区
 - **回滚日志**：实现数据回滚时对数据的恢复，MVCC实现的重要组成
- 默认大小12M，文件名称ibdata1，配置参数：`innodb_data_file_path`
- 系统表空间会自动增长，每次增量64MB，且增长之后的文件不可缩减。

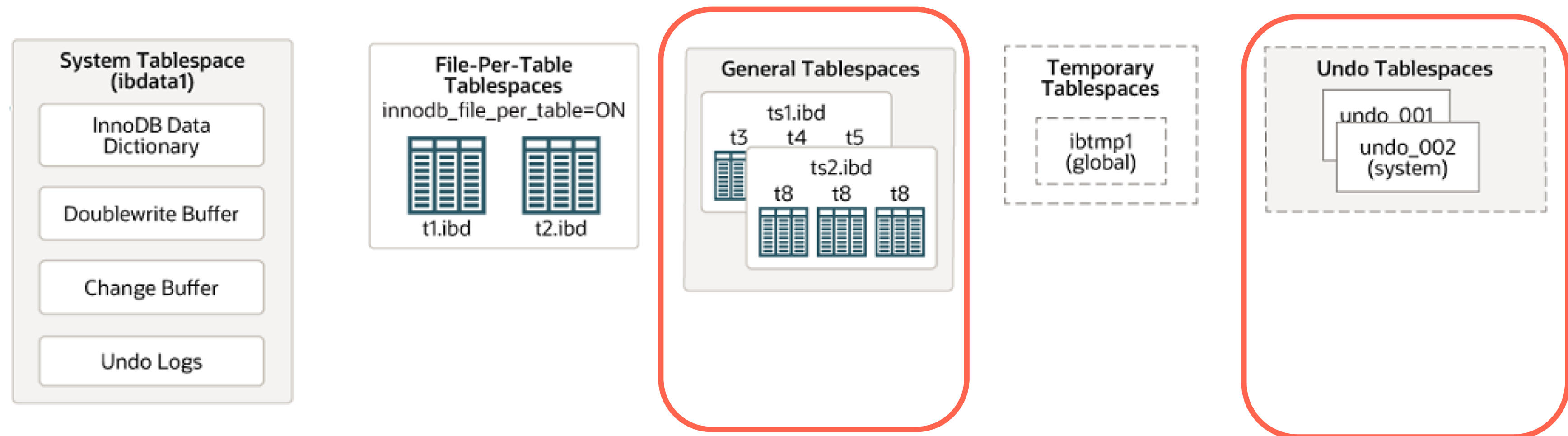
2.4 InnoDB-磁盘结构



02-独立表空间（File-per-table Tablespace）

- 独立表空间用于存放每个表的数据和索引。在5.7版本中默认开启，初始化大小是96KB
- 其他类型的信息，如：回滚日志、双写缓冲区、系统事务信息、修改缓冲等仍存放于系统表空间内，因此即使用了独立表空间，系统表空间也会不断增长。
- 开启关闭： `innodb_file_per_table=ON|OFF`
- 在数据库文件夹内，为每张表单独创建表空间 **表名.ibd** 文件存数据与索引。同时创建 **表名.frm** 文件存表结构信息

2.4 InnoDB-磁盘结构



03-通用表空间（General Tablespace）

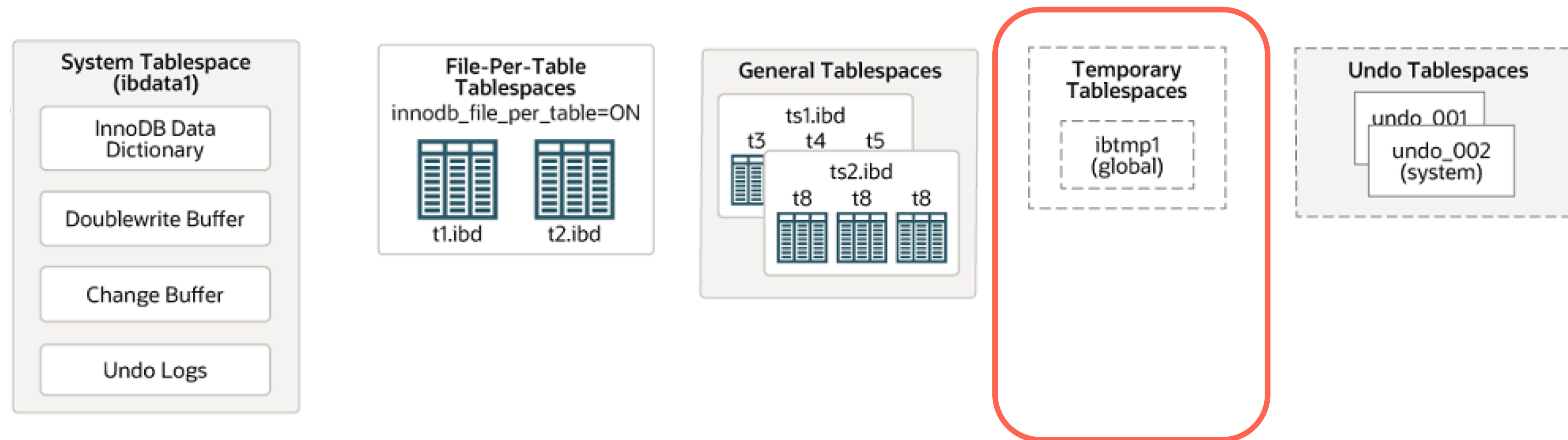
- 通用表空间存在的目的是为了在系统表空间与独立表空间之间作出平衡
- 系统表空间与独立表空间中的表可以向通用表空间移动，反之亦可
- 但系统表空间中的表无法直接与独立表空间中的表相互转化。

04-回滚表空间（Undo Tablespace）

- Undo TableSpace 用于存放一个或多个 undo log 文件，默认大小为10MB
- Undo log默认存储在系统表空间，在5.7版本中支持自定义到独立的表空间



2.4 InnoDB-磁盘结构

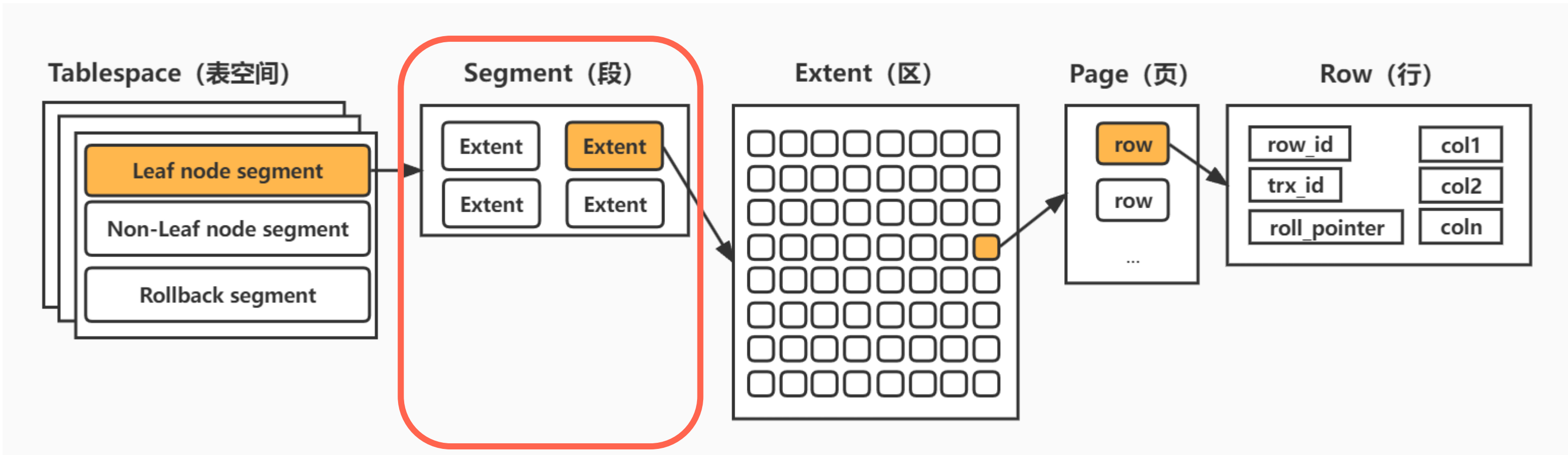


05-临时表空间 (The Temporary Tablespace)

- MySQL 5.7 之前临时表存储在系统表空间，这样会导致ibdata在使用临时表的场景下疯狂增长
- MySQL 5.7 版本里InnoDB引擎从系统表空间中抽离出临时表空间独立保存临时表数据
- 配置属性: `innodb_temp_data_file_path`



2.4 InnoDB-磁盘结构-存储结构



表空间由段（Segment）、区（extent）、页（Page）、行（Row）组成

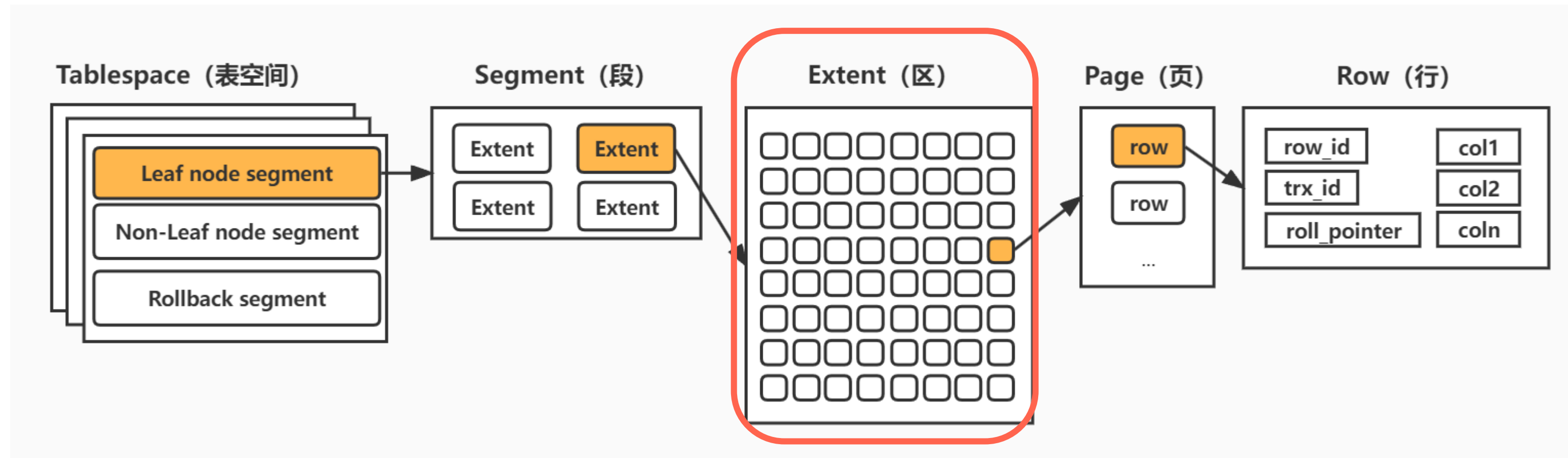
01-段（Segment）

- 表空间由各个段组成，段类型分数据段、索引段、回滚段。
- MySQL的索引数据结构是B+树，这个树有叶子节点和非叶子节点
- 一个段包含多个区，至少有一个区，段扩展的最小单位是区
 - 数据段是叶子节点 Leaf node segment
 - 索引段是非叶子节点 Non-Leaf node segment



为啥要用B+树？用红黑树行不行？

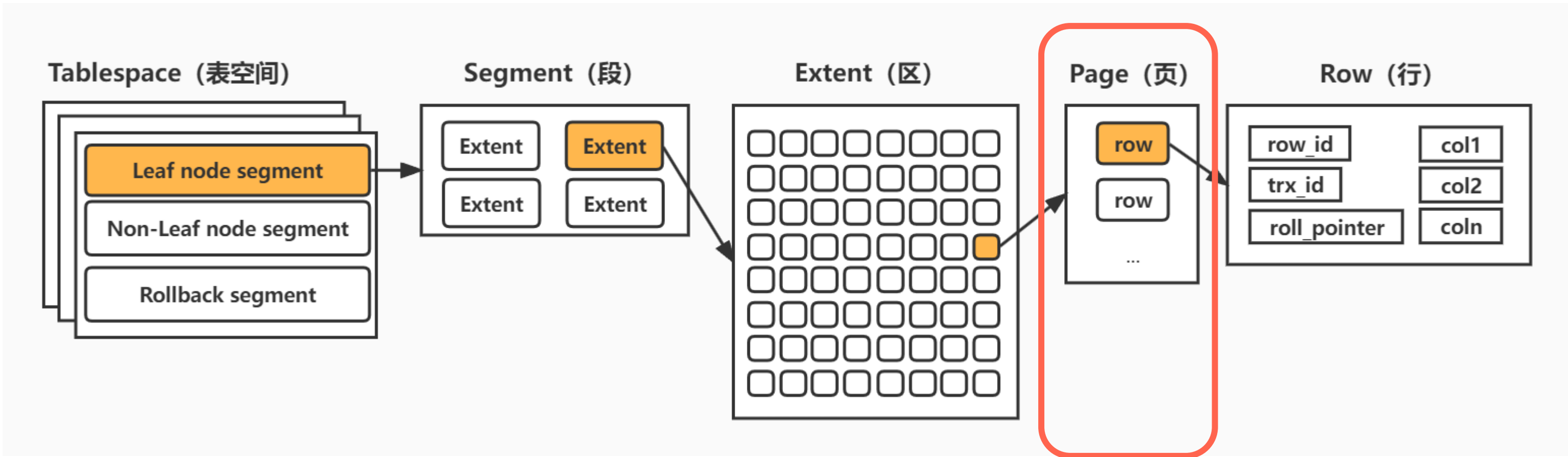
2.4 InnoDB-磁盘结构-存储结构



02-区 (Extent)

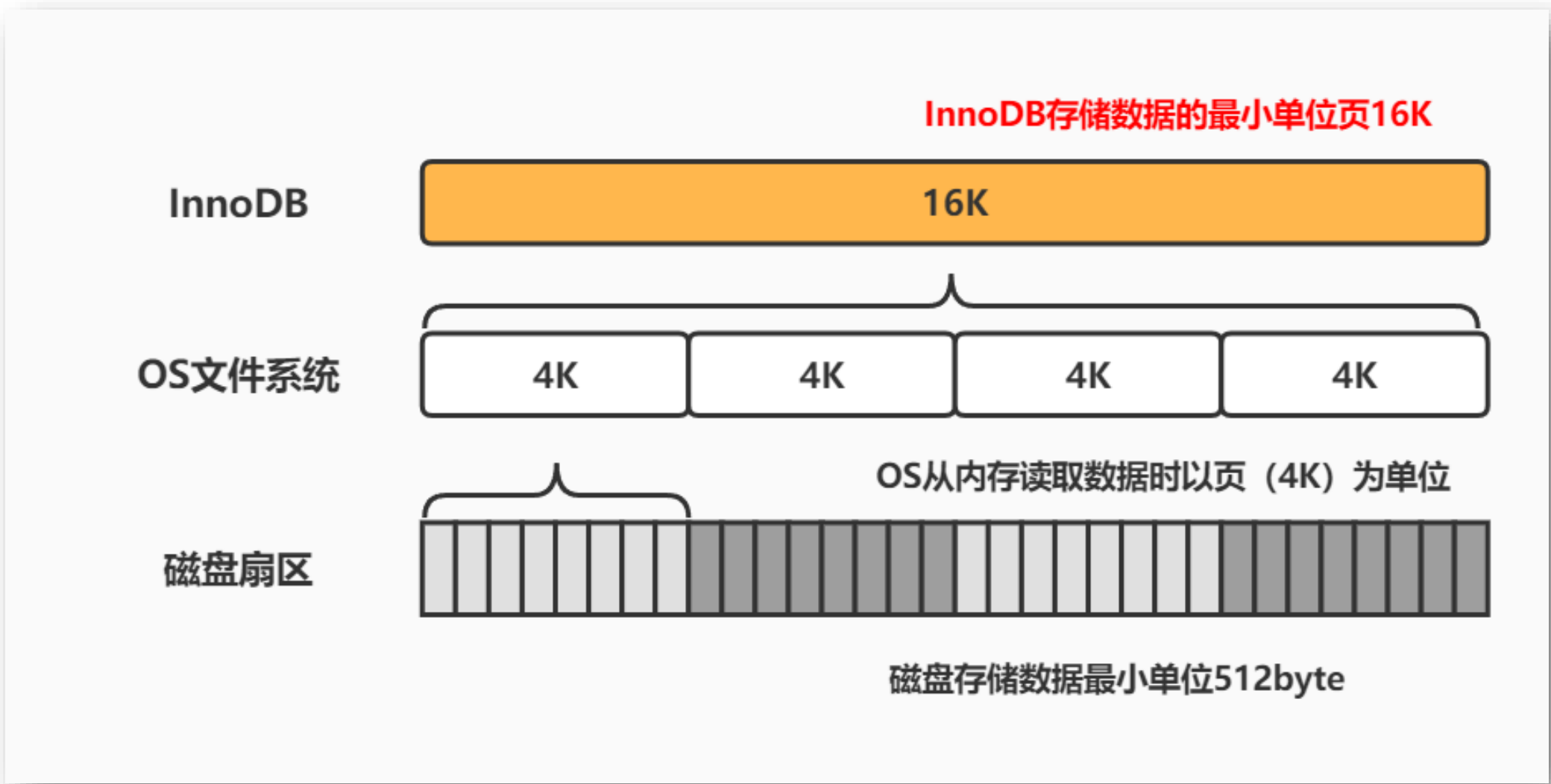
- 区是由连续的页组成的空间，大小固定为 1MB
- 默认情况下，一个区里有64个页
- 为了保证区的连续性，InnoDB一次会从磁盘申请4-5个区

2.4 InnoDB-磁盘结构-存储结构

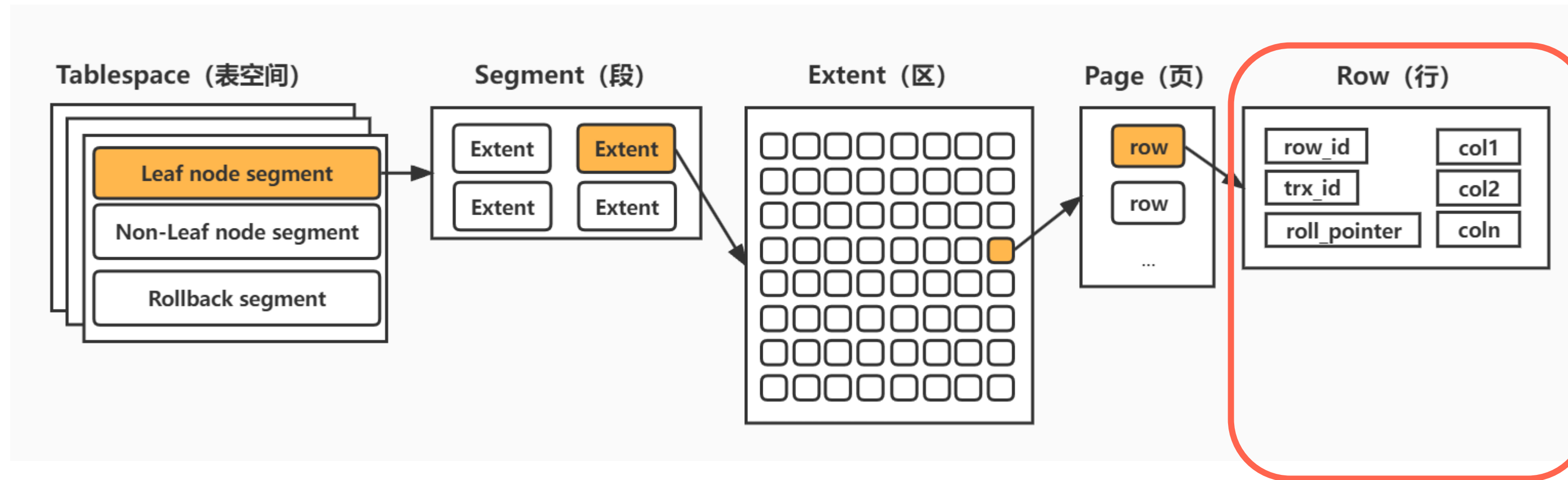


03-页 (Page)

- **页是 InnoDB 的基本存储单位**，页默认大小是16K（可配置innodb_page_size），InnoDB 首次加载后便无法更改
- **操作系统读写磁盘最小单位是页，4K**
- **磁盘存储数据量最小单位512 byte**



2.4 InnoDB-磁盘结构-存储结构

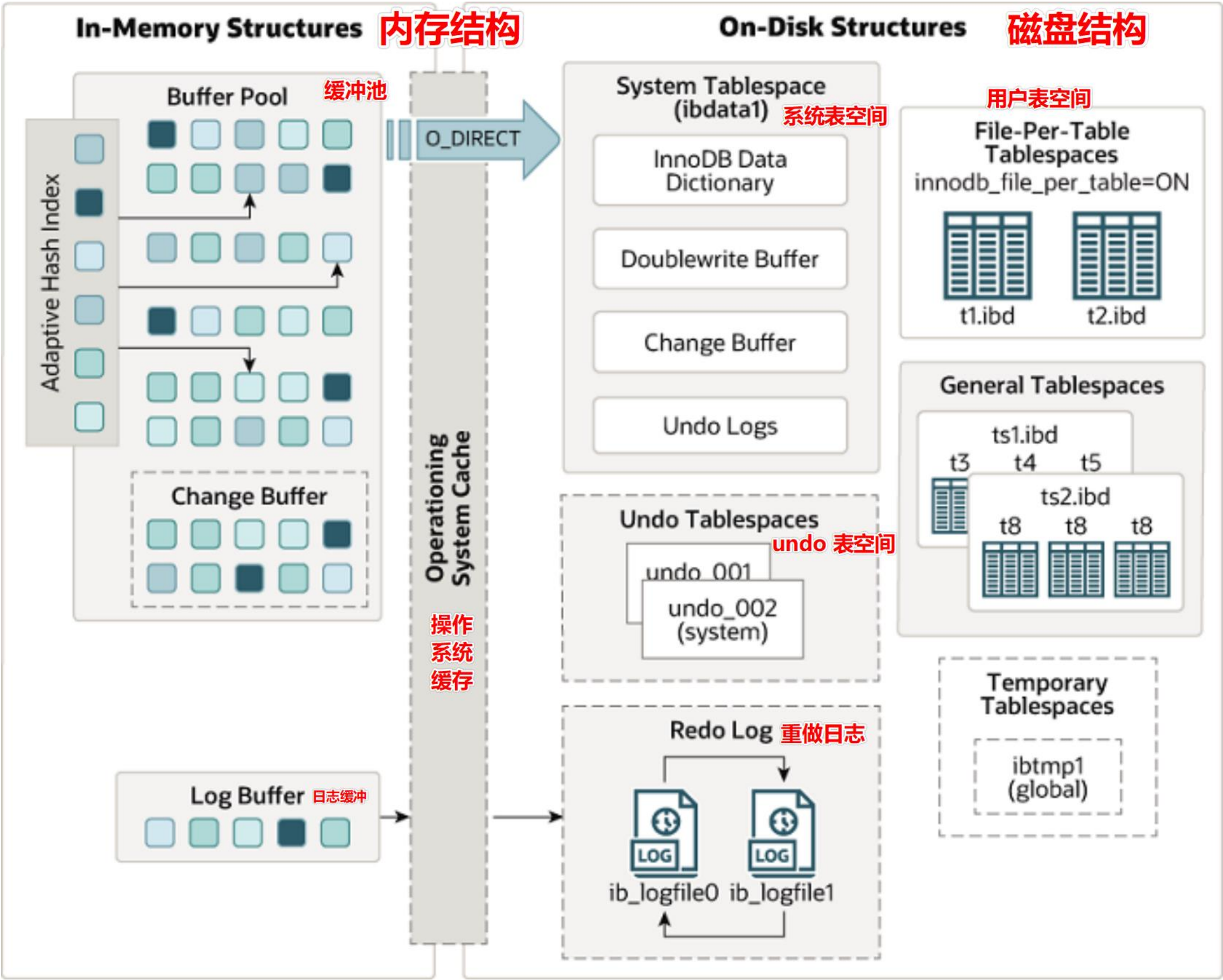


04-行 (Row)

- InnoDB的数据是以行为单位存储，一个页中包含多个行
- InnoDB提供4种行格式：Compact、Redundant、Dynamic和Compressed
- 默认行格式Dynamic



2.5 内存中的数据如何进入磁盘？



这个问题蛮重要！

2.6 脏页落盘

01-什么是脏页？

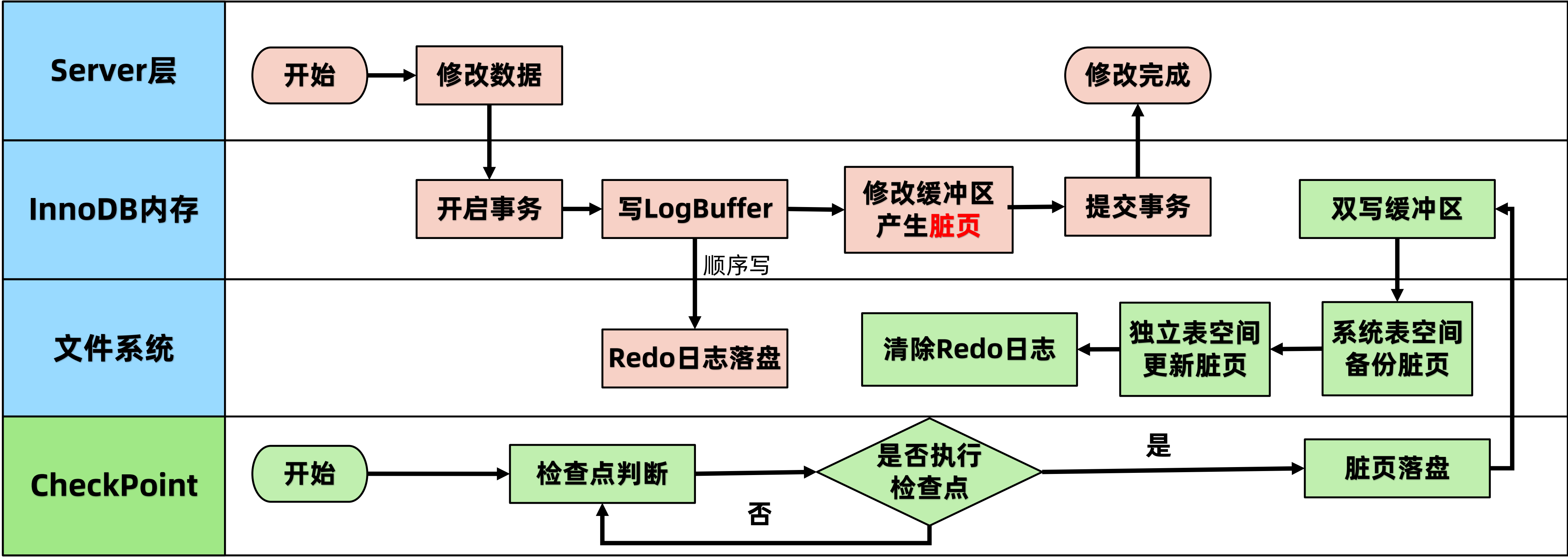
- 对数据的修改操作，首先修改**内存结构**中缓冲区中的页，缓冲区中的页与**磁盘**中的页数据不一致，所以称缓冲区中的页为**脏页**。

02-脏页如何进入到磁盘？

- 脏页从缓冲区刷新到磁盘，不是每次页更新之后触发，而是通过**CheckPoint机制**刷新磁盘
- 脏页落盘涉及的问题很多，咱们会重点分析，首先咱们来看下数据整体落盘流程

2.6 脏页落盘

03-InnoDB数据落盘流程



灵魂三问：如何提升性能？如何持久化数据？安全性怎么保证？

2.6 脏页落盘

04-如何提升性能？

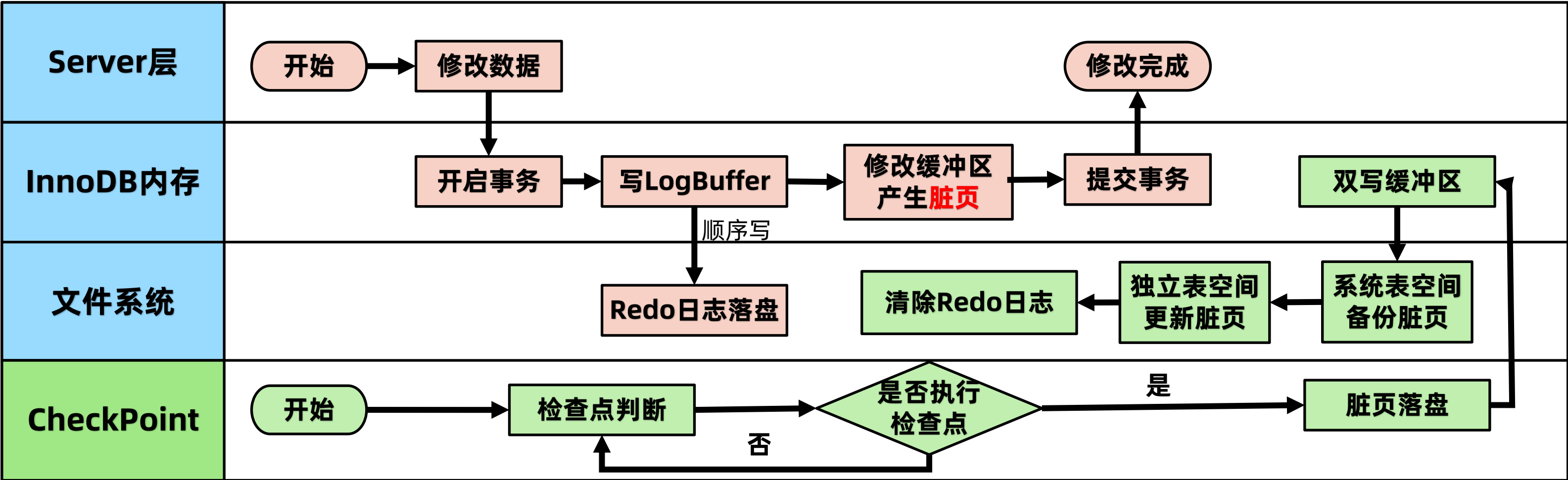
- 内存中写，操作过程记录日志，日志批量写入磁盘
- 分散写变顺序写

05-如何持久化数据？

- 通过CheckPoint机制进行脏页落盘
- 日志先行，所有操作前先写Redo日志

06-数据安全性怎么保证？

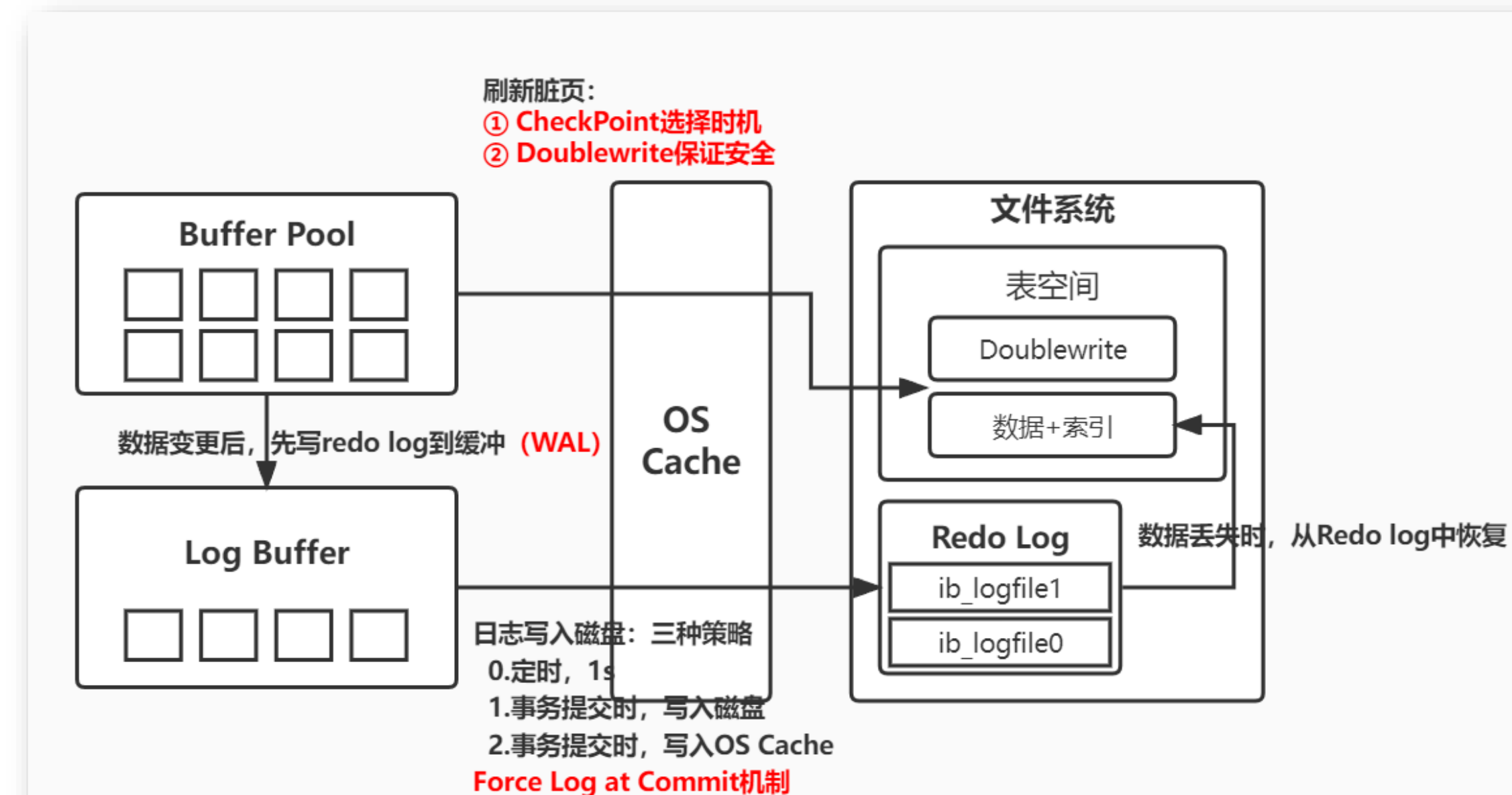
- Force Log at Commit
- Write Ahead Log (WAL)
- CheckPoint机制
- Double Write机制



2.6 脏页落盘

07-为什么不是每次更新直接写入磁盘？

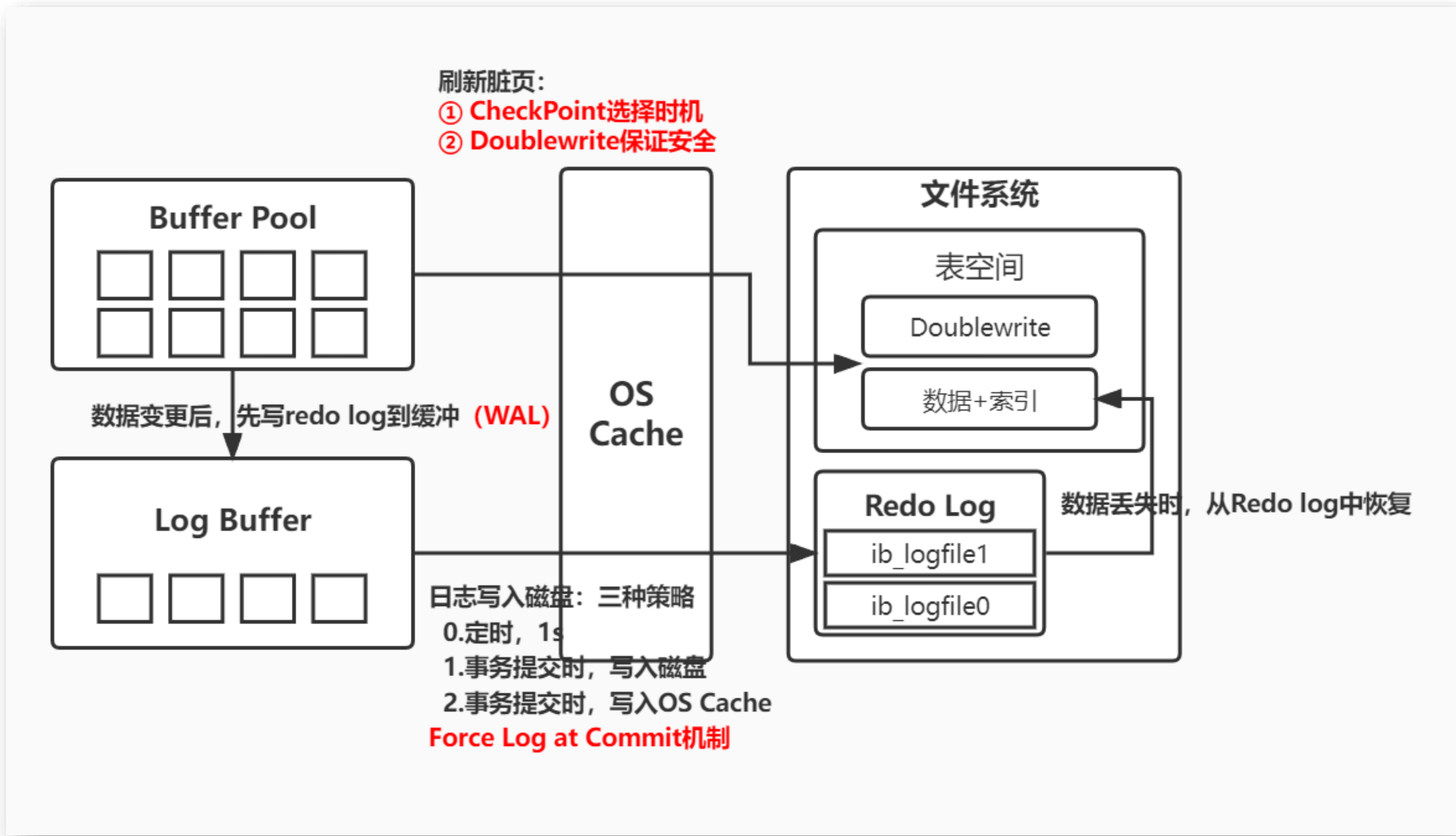
- 如果每次页发生变化就落盘，一个页落盘必然伴随4次IO操作，性能开销很大，而且随着写入操作增加性能开销是指数级增长
- 当然，数据也不能在内存中保存太长时间，**时间越久安全性风险越高**
- InnoDB采用**Write Ahead Log**策略和Force Log at Commit机制实现事务的持久性
 - **Write Ahead Log：日志先行**，数据变更写入磁盘前，必须将内存中的日志写入到磁盘
 - **Force Log at Commit**：当事务提交时，所有事务产生的日志都必须刷到磁盘



2.6 脏页落盘

08-怎么确保日志安全进入磁盘？

- 为了确保日志写入到磁盘，将redo日志写入Log Buffer后调用fsync函数，将缓冲日志文件从OS Cache中写入磁盘
- 日志进入磁盘不仅要考虑安全性，还需兼顾性能！



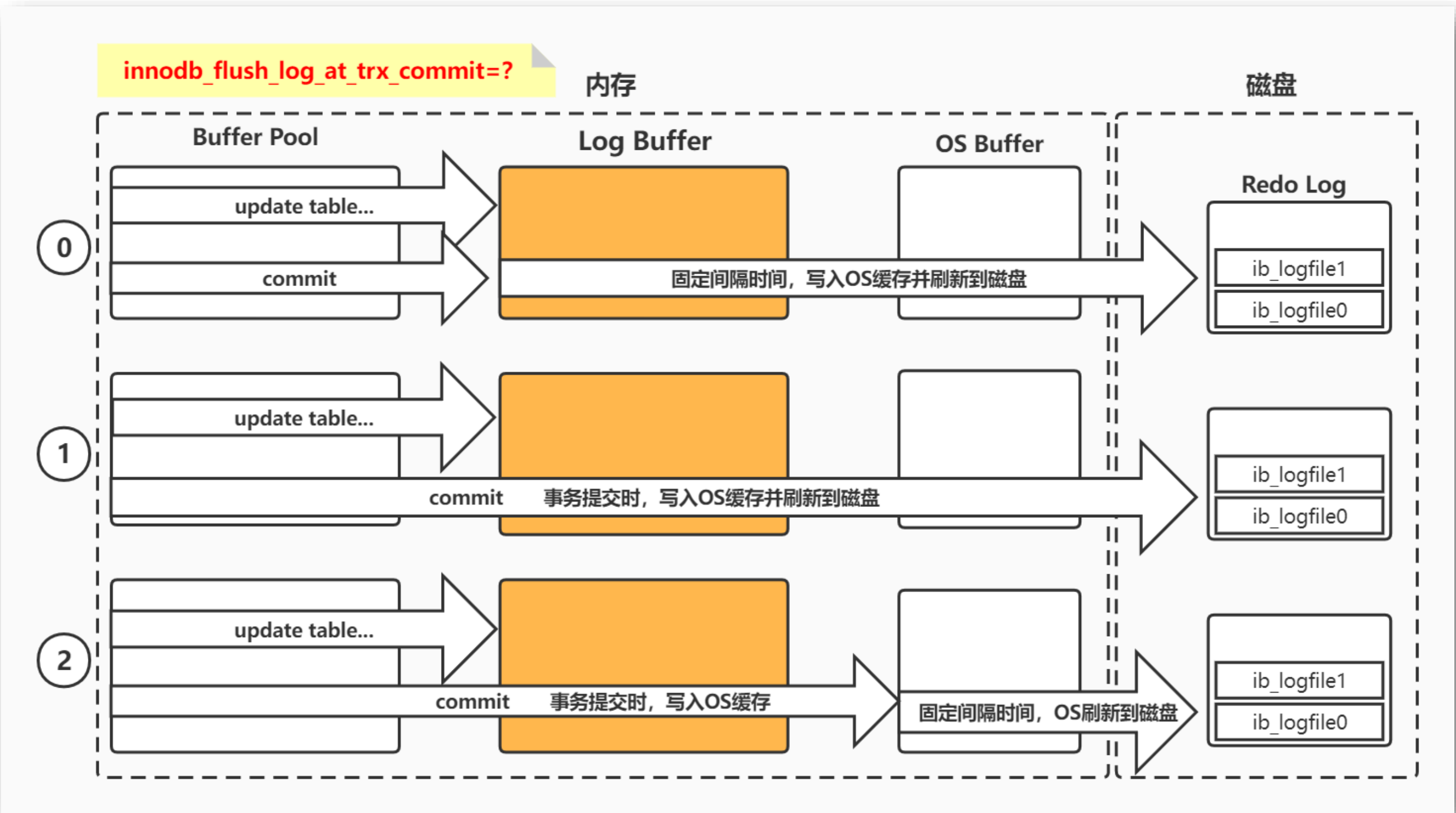
事务提交时，redo日志刷盘是最佳策略吗？

2.7 Redo日志落盘

Redo日志默认落盘策略，事务提交立即落盘。Log Buffer写入磁盘的时机由参数 innodb_flush_log_at_trx_commit 控制，此参数控制每次事务提交时InnoDB的行为。

三个配置：

- 为0时：每秒写入，与事务无关
 - 最多丢失1秒的事务操作
 - 写入效率最高，安全性最低
- 为1时：事务提交，写入磁盘
 - 不会丢失数据
 - 写入效率最低，安全性最高
- 为2时：事务提交，写入OS Buffer
 - 数据安全性依赖于系统，最多丢1秒事务操作
 - 写入效率居中，安全性居中



2.8 CheckPoint机制

01-什么是CheckPoint机制？

- 它是将缓冲池中的脏页数据刷到磁盘上的机制，决定脏页落盘的时机、条件和脏页的选择等
- CheckPoint的类型不止一种

02-解决什么问题？

- **脏页落盘：**避免数据更改直接操作磁盘
- **缩短数据库的恢复时间：**数据库宕机时，不用重做所有redo日志，大大缩短恢复时间
- **缓冲池不够用时，将脏页刷新到磁盘：**Buffer Pool不够用时溢出页落盘，LRU淘汰掉的非热点数据
- **redo日志不可用时，刷新脏页：**日志文件可以循环使用，不会无限增长

03-分类：

- **sharp checkpoint：**关闭数据库时将脏页全部刷新到磁盘中
- **fuzzy checkpoint：**默认方式，在运行时选择不同时机将脏页刷盘，只刷新部分脏页

2.8 CheckPoint机制

03-分类:

- **sharp checkpoint:** 关闭数据库时将脏页全部刷新到磁盘中
- **fuzzy checkpoint:** 默认方式，在运行时选择不同时机将脏页刷盘。只刷新部分脏页
 - **Master Thread Checkpoint:** 固定频率刷新部分脏页到磁盘，异步操作不会阻塞用户线程
 - **FLUSH_LRU_LIST Checkpoint:** 缓冲池淘汰非热点Page，如果该Page是脏页会执行CheckPoint
 - **Async/Sync Flush Checkpoint:** redo日志不可用时，强制脏页落盘，有了前两个这种一般不会发生
 - **Dirty Page too much Checkpoint:** 脏页占比太多强制进行刷盘，阈值75%

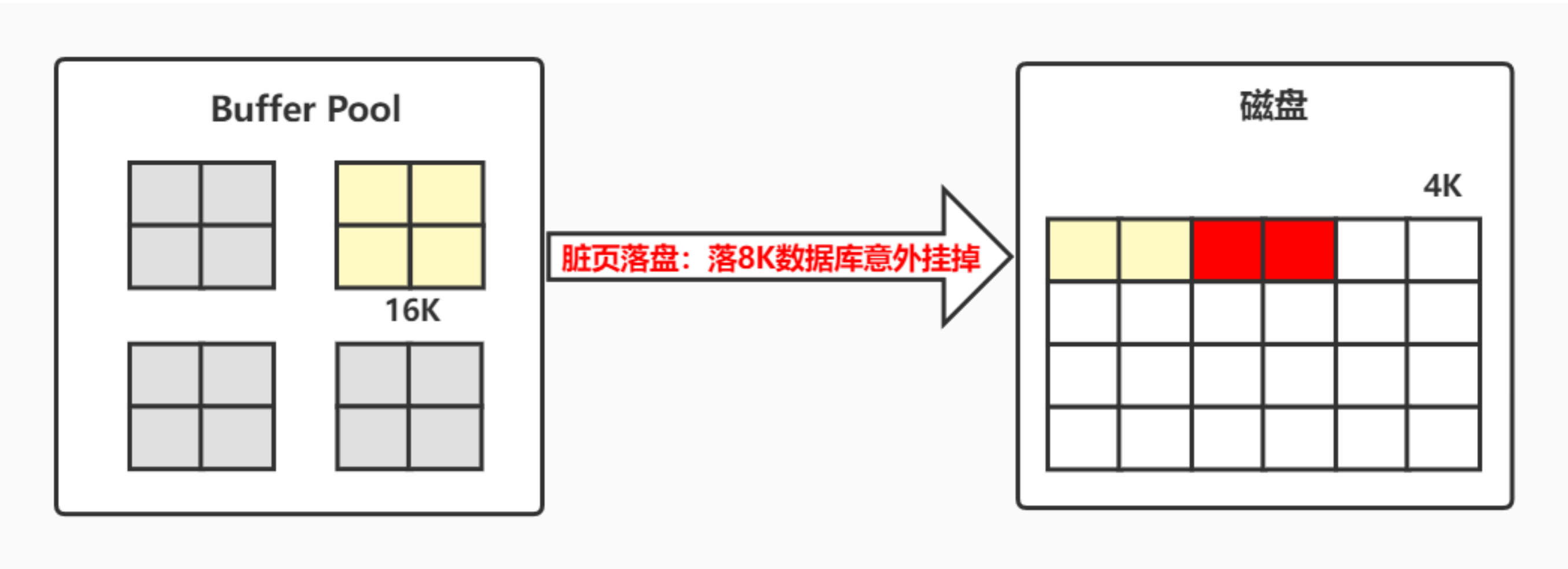


脏页大小16K，如果只写入了8K系统就故障了，会怎么样？

2.9 Double Write机制

01-写失效问题

数据库准备刷新脏页时，将16KB的刷入磁盘，但当写入了8KB时，就宕机了这种情况只写了部分没完成的情况被称为**写失效Partial Page Write**



写失效了可以通过redo日志恢复么？

2.9 Double Write机制

02-Double Write机制

➤ **Double Write**其实就是写两次，在修改记录redo日志前，先做个副本留个“备胎”

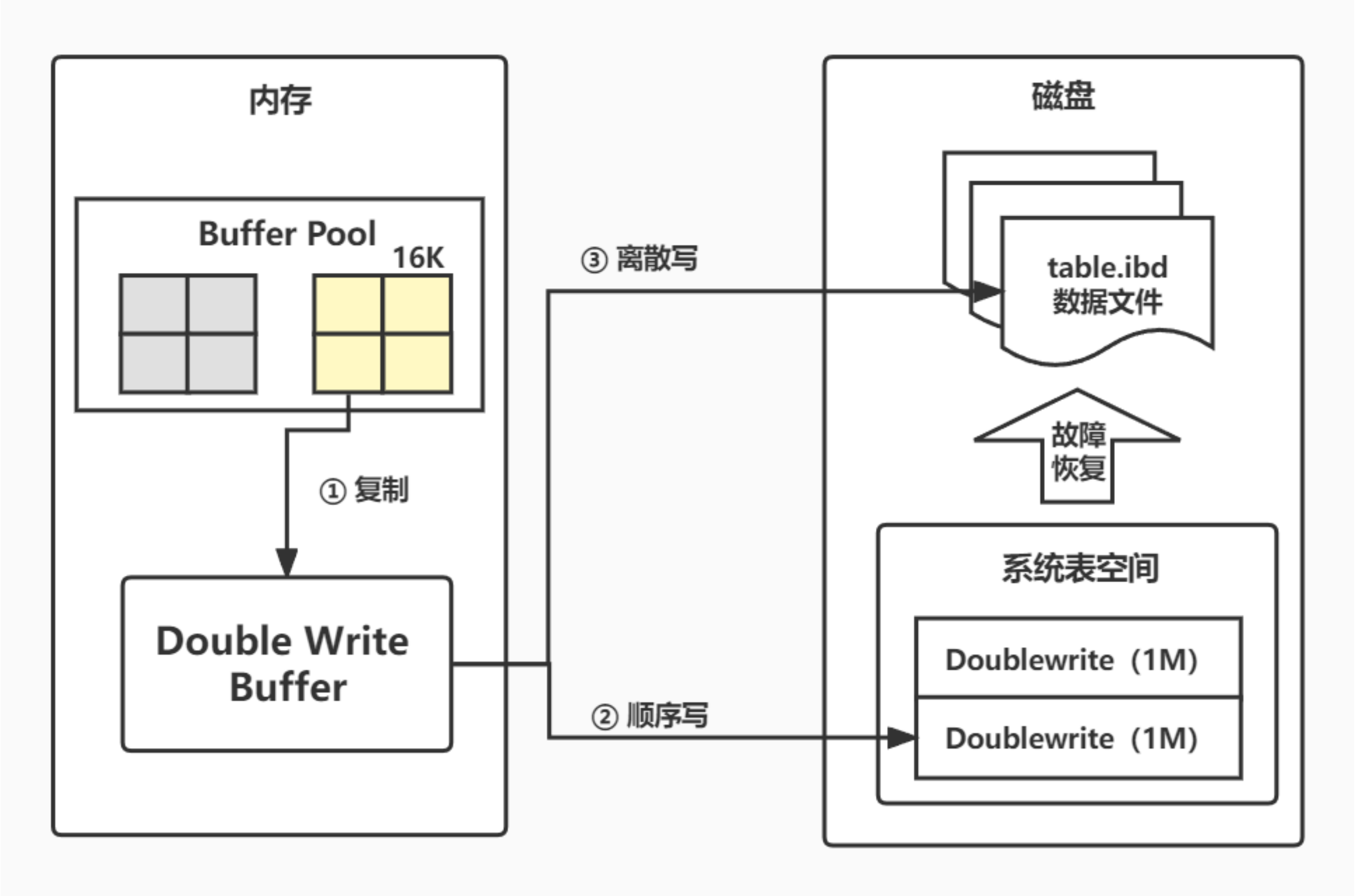
注意：Redo日志不能解决写失效问题，因为redo日志记录的是对页的修改记录而不是数据本身

两个部分：

- 内存中，大小为2MB
- 磁盘系统表空间中，也为2MB，2个区，连续128个页

流程：

- ① 首先复制
- ② 在顺序写入磁盘第一次
- ③ 最后离散写入磁盘第二次

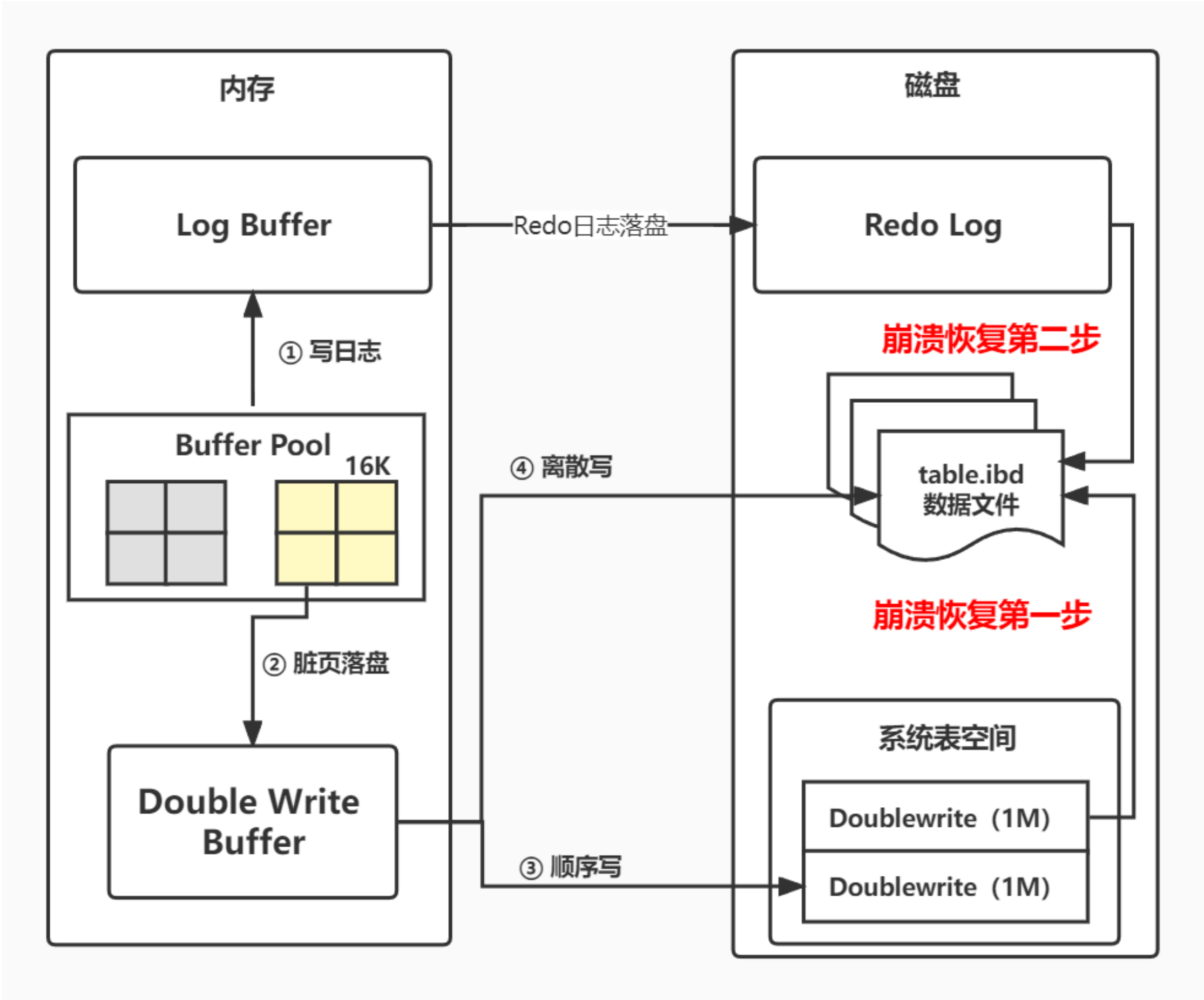


2.9 Double Write机制

03-Double Write崩溃恢复过程

- ① 首先找到系统表空间中Double Write区域对应的页副本数据
- ② 然后将其复制到独立表空间
- ③ 最后清楚redo日志

-💡- 数据落袋为安了，再思考一个问题，事务回滚的是什么数据？



THANKS

 极客时间 | 训练营

教育不是注满一桶水，而是点燃一把火